Java Questions

http://java-questions.com/ImmutableClass-interview-questions.html

1) What is Java?

Java is the high-level, object-oriented, robust, secure programming language, platform-independent, high performance, Multithreaded, and portable programming language. It was developed by **James Gosling** in June 1991. It can also be known as the platform as it provides its own JRE and API.

2) What are the differences between C++ and Java?

Comparison Index	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Mainly used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in window, web-based, enterprise and mobile applications.
Design Goal	C++ was designed for systems and applications programming. It was an extension of C programming language.	Java was designed and created as an interpreter for printing systems but later extended as a support network computing. It was designed with a goal of being easy to use and accessible to a broader audience.
Goto	C++ supports the goto statement.	Java doesn't support the goto statement.
Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by interfaces in java.
Operator Overloading	C++ supports operator overloading.	Java doesn't support operator overloading.
Pointers	C++ supports pointers. You can write pointer program in C++.	Java supports pointer internally. However, you can't write the pointer program in java. It means java has restricted pointer support in java.
Compiler and Interpreter	C++ uses compiler only. C++ is compiled and run using the compiler which converts source code into machine code so, C++ is platform dependent.	Java uses compiler and interpreter both. Java source code is converted into bytecode at compilation time. The interpreter executes this bytecode at runtime and produces output. Java is interpreted that is why it is platform independent.
Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.
Structure and Union	C++ supports structures and unions.	Java doesn't support structures and unions.
Thread Support	C++ doesn't have built-in support for threads. It relies on third-party libraries for thread support.	Java has built-in thread support.
Documentation comment	C++ doesn't support documentation comment.	Java supports documentation comment (/** ... */) to create documentation for java source code.
Virtual Keyword	C++ supports virtual keyword so that we can decide whether or not override a function.	Java has no virtual keyword. We can override all non-static methods by default. In other words, non-static methods are virtual by default.
unsigned right shift >>>	C++ doesn't support >>> operator.	Java supports unsigned right shift >>> operator that fills zero at the top for the negative numbers. For positive numbers, it works same like >> operator.
Inheritance Tree	C++ creates a new inheritance tree always.	Java uses a single inheritance tree always because all classes are the child of Object class in java. The object class is the root of the inheritance tree in java.
Hardware	C++ is nearer to hardware.	Java is not so interactive with hardware.
Object-oriented	C++ is an object-oriented language. However, in C language, single root hierarchy is not possible.	Java is also an object-oriented language. However, everything (except fundamental types) is an object in Java. It is a single root hierarchy as everything gets derived from java.lang.Object.

---

3) List the features of Java Programming language.

There are the following features in Java Programming Language.

Simple: Java is easy to learn. The syntax of Java is based on C++ which makes easier to write the program in it.

Object-Oriented: Java follows the object-oriented paradigm which allows us to maintain our code as the combination of different type of objects that incorporates both data and behavior.

Portable: Java supports read-once-write-anywhere approach. We can execute the Java program on every machine. Java program (.java) is converted to bytecode (.class) which can be easily run on every machine.

Platform Independent: Java is a platform independent programming language. It is different from other programming languages like C and C++ which needs a platform to be executed. Java comes with its platform on which its code is executed. Java doesn't depend upon the operating system to be executed.

Secured: Java is secured because it doesn't use explicit pointers. Java also provides the concept of ByteCode and Exception handling which makes it more secured.

Robust: Java is a strong programming language as it uses strong memory management. The concepts like Automatic garbage collection, Exception handling, etc. make it more robust.

Architecture Neutral: Java is architectural neutral as it is not dependent on the architecture. In C, the size of data types may vary according to the architecture (32 bit or 64 bit) which doesn't exist in Java.

Interpreted: Java uses the Just-in-time (JIT) interpreter along with the compiler for the program execution.

High Performance: Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code. It is still a little bit slower than a compiled language (e.g., C++).

Multithreaded: We can write Java programs that deal with many tasks at once by defining multiple threads. The main advantage of multi-threading is that it doesn't occupy memory for each thread. It shares a common memory area. Threads are important for multi-media, Web applications, etc.

Distributed: Java is distributed because it facilitates users to create distributed applications in Java. RMI and EJB are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.
Dynamic: Java is a dynamic language. It supports dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

---

4) What do you understand by Java virtual machine?
Java Virtual Machine is a virtual machine that enables the computer to run the Java program. JVM acts like a run-time engine which calls the main method present in the Java code. JVM is the specification which must be implemented in the computer system. The Java code is compiled by JVM to be a Bytecode which is machine independent and close to the native code.
5) What is the difference between JDK, JRE, and JVM?

JVM: JVM is an acronym for Java Virtual Machine; it is an abstract machine which provides the runtime environment in which Java bytecode can be executed. It is a specification which specifies the working of Java Virtual Machine. Its implementation has been provided by Oracle and other companies. Its implementation is known as JRE.
JVMs are available for many hardware and software platforms (so JVM is platform dependent). It is a runtime instance which is created when we run the Java class. There are three notions of the JVM: specification, implementation, and instance.
JRE: JRE stands for Java Runtime Environment. It is the implementation of JVM. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment. It is the implementation of JVM. It physically exists. It contains a set of libraries + other files that JVM uses at runtime.
JDK: JDK is an acronym for Java Development Kit. It is a software development environment which is used to develop Java applications and applets. It physically exists. It contains JRE + development tools. JDK is an implementation of any one of the below given Java Platforms released by Oracle Corporation:
Standard Edition Java Platform
Enterprise Edition Java Platform
Micro Edition Java Platform

6) How many types of memory areas are allocated by JVM?
Class(Method) Area: Class Area stores per-class structures such as the runtime constant pool, field, method data, and the code for methods.
Heap: It is the runtime data area in which the memory is allocated to the objects
Stack: Java Stack stores frames. It holds local variables and partial results, and plays a part in method invocation and return. Each thread has a private JVM stack, created at the same time as the thread. A new frame is created each time a method is invoked. A frame is destroyed when its method invocation completes.
Program Counter Register: PC (program counter) register contains the address of the Java virtual machine instruction currently being executed.

Native Method Stack: It contains all the native methods used in the application.

---

7) What is JIT compiler?
Just-In-Time(JIT) compiler: It is used to improve the performance. JIT compiles parts of the bytecode that have similar functionality at the same time, and hence reduces the amount of time needed for compilation. Here the term “compiler” refers to a translator from the instruction set of a Java virtual machine (JVM) to the instruction set of a specific CPU.

---

8) What is the platform?

A platform is the hardware or software environment in which a piece of software is executed. There are two types of platforms, software-based and hardware-based. Java provides the software-based platform.

---

9) What are the main differences between the Java platform and other platforms?

There are the following differences between the Java platform and other platforms.
Java is the software-based platform whereas other platforms may be the hardware platforms or software-based platforms.
Java is executed on the top of other hardware platforms whereas other platforms can only have the hardware components.

---

10) What gives Java its 'write once and run anywhere' nature?

The bytecode. Java compiler converts the Java programs into the class file (Byte Code) which is the intermediate language between source code and machine code. This bytecode is not platform specific and can be executed on any computer.

---

11) What is classloader?

Classloader is a subsystem of JVM which is used to load class files. Whenever we run the java program, it is loaded first by the classloader. There are three built-in classloaders in Java.

Bootstrap ClassLoader: This is the first classloader which is the superclass of Extension classloader. It loads the *rt.jar* file which contains all class files of Java Standard Edition like java.lang package classes, java.net package classes, java.util package classes, java.io package classes, java.sql package classes, etc.

Extension ClassLoader: This is the child classloader of Bootstrap and parent classloader of System classloader. It loads the jar files located inside *$JAVA_HOME/jre/lib/ext* directory.

System/Application ClassLoader: This is the child classloader of Extension classloader. It loads the class files from the classpath. By default, the classpath is set to the current directory. You can change the classpath using "-cp" or "-classpath" switch. It is also known as Application classloader.

---

12) Is Empty .java file name a valid source file name?

Yes, Java allows to save our java file by **.java** only, we need to compile it by **javac .java** and run by **java classname** Let's take a simple example:

```
//save by .java only
class A{
public static void main(String args[]){
System.out.println("Hello java");
}
}
//compile by javac .java
//run by    java A
```

compile it by **javac .java**
run it by **java A**

---

13) Is delete, next, main, exit or null keyword in java?
No.

---

14) If I don't provide any arguments on the command line, then what will the value stored in the String array passed into the main() method, empty or NULL?
It is empty, but not null.

---

15) What if I write static public void instead of public static void?
The program compiles and runs correctly because the order of specifiers doesn't matter in Java.

16) What is the default value of the local variables?
The local variables are not initialized to any default value, neither primitives nor object references.

17) What are the various access specifiers in Java?
In Java, access specifiers are the keywords which are used to define the access scope of the method, class, or a variable. In Java, there are four access specifiers given below.
Public The classes, methods, or variables which are defined as public, can be accessed by any class or method.
Protected Protected can be accessed by the class of the same package, or by the sub-class of this class, or within the same class.
Default Default are accessible within the package only. By default, all the classes, methods, and variables are of default scope.
Private The private class, methods, or variables defined as private can be accessed within the class only.

18) What is the purpose of static methods and variables?

The methods or variables defined as static are shared among all the objects of the class. The static is the part of the class and not of the object. The static variables are stored in the class area, and we do not need to create the object to access such variables. Therefore, static is used in the case, where we need to define variables or methods which are common to all the objects of the class.
For example, In the class simulating the collection of the students in a college, the name of the college is the common attribute to all the students. Therefore, the college name will be defined as **static**.

19) What are the advantages of Packages in Java?
There are various advantages of defining packages in Java.
Packages avoid the name clashes.
The Package provides easier access control.
We can also have the hidden classes that are not visible outside and used by the package.
It is easier to locate the related classes.

20) What is the output of the following Java program?

```
class Test
{
  public static void main (String args[])
  {
    System.out.println(10 + 20 + "Javatpoint");
    System.out.println("Javatpoint" + 10 + 20);
  }
}
```

The output of the above code will be
30Javatpoint
Javatpoint1020

Explanation
In the first case, 10 and 20 are treated as numbers and added to be 30. Now, their sum 30 is treated as the string and concatenated with the string **Javatpoint**. Therefore, the output will be **30Javatpoint**.
In the second case, the string Javatpoint is concatenated with 10 to be the string **Javatpoint10** which will then be concatenated with 20 to be **Javatpoint1020**.

21) What is the output of the following Java program?

```
class Test
{
  public static void main (String args[])
  {
    System.out.println(10 * 20 + "Javatpoint");
    System.out.println("Javatpoint" + 10 * 20);
  }
}
```

The output of the above code will be
200Javatpoint
Javatpoint200
Explanation

In the first case, The numbers 10 and 20 will be multiplied first and then the result 200 is treated as the string and concatenated with the string **Javatpoint** to produce the output **200Javatpoint**.
In the second case, The numbers 10 and 20 will be multiplied first to be 200 because the precedence of the multiplication is higher than addition. The result 200 will be treated as the string and concatenated with the string **Javatpoint**to produce the output as **Javatpoint200**.

---

22) What is the output of the following Java program?

```
class Test
{
  public static void main (String args[])
  {
    for(int i=0; 0; i++)
    {
      System.out.println("Hello Javatpoint");
    }
  }
}
```

The above code will give the compile-time error because the for loop demands a boolean value in the second part and we are providing an integer value, i.e., 0.

Core Java - OOPs Concepts: Initial OOPs Interview Questions

23) What is object-oriented paradigm?
It is a programming paradigm based on objects having data and methods defined in the class to which it belongs. Object-oriented paradigm aims to incorporate the advantages of modularity and reusability. Objects are the instances of classes which interacts with one another to design applications and programs. There are the following features of the object-oriented paradigm.
Follows the bottom-up approach in program design.
Focus on data with methods to operate upon the object's data
Includes the concept like Encapsulation and abstraction which hides the complexities from the user and show only functionality.
Implements the real-time approach like inheritance, abstraction, etc.
The examples of the object-oriented paradigm are C++, Simula, Smalltalk, Python, C#, etc.

---

24) What is an object?
The Object is the real-time entity having some state and behavior. In Java, Object is an instance of the class having the instance variables as the state of the object and the methods as the behavior of the object. The object of a class can be created by using the **new** keyword.

---

25) What is the difference between an object-oriented programming language and object-based programming language?
There are the following basic differences between the object-oriented language and object-based language.
Object-oriented languages follow all the concepts of OOPs whereas, the object-based language doesn't follow all the concepts of OOPs like inheritance and polymorphism.
Object-oriented languages do not have the inbuilt objects whereas Object-based languages have the inbuilt objects, for example, JavaScript has window object.
Examples of object-oriented programming are Java, C#, Smalltalk, etc. whereas the examples of object-based languages are JavaScript, VBScript, etc.

---

26) What will be the initial value of an object reference which is defined as an instance variable?
All object references are initialized to null in Java.
Core Java - OOPs Concepts: Constructor Interview Questions
27) What is the constructor?
The constructor can be defined as the special type of method that is used to initialize the state of an object. It is invoked when the class is instantiated, and the memory is allocated for the object. Every time, an object is created using the **new** keyword, the default constructor of the class is called. The name of the constructor must be similar to the class name. The constructor must not have an explicit return type.

---

28) How many types of constructors are used in Java?
Based on the parameters passed in the constructors, there are two types of constructors in Java.
Default Constructor: default constructor is the one which does not accept any value. The default constructor is mainly used to initialize the instance variable with the default values. It can also be used for performing some useful task on object creation. A default constructor is invoked implicitly by the compiler if there is no constructor defined in the class.
Parameterized Constructor: The parameterized constructor is the one which can initialize the instance variables with the given values. In other words, we can say that the constructors which can accept the arguments are called parameterized constructors.
29) What is the purpose of a default constructor?

The purpose of the default constructor is to assign the default value to the objects. The java compiler creates a default constructor implicitly if there is no constructor in the class.

```
class Student3{
int id;
String name;
void display(){System.out.println(id+" "+name);}
public static void main(String args[]){
Student3 s1=new Student3();
Student3 s2=new Student3();
s1.display();  s2.display();
} }
```

Output:0 null 0 null

Explanation: In the above class, you are not creating any constructor, so compiler provides you a default constructor. Here 0 and null values are provided by default constructor.

---

30) Does constructor return any value?

Ans: yes, The constructor implicitly returns the current instance of the class (You can't use an explicit return type with the constructor). More Details.

---

31)Is constructor inherited?

No, The constructor is not inherited.

---

32) Can you make a constructor final?

No, the constructor can't be final.

---

33) Can we overload the constructors?

Yes, the constructors can be overloaded by changing the number of arguments accepted by the constructor or by changing the data type of the parameters. Consider the following example.

```
class Test
{
  int i;
  public Test(int k)
  {       i=k;    }
  public Test(int k, int m)
  {
    System.out.println("Hi I am assigning the value max(k, m) to i");
    if(k>m)
    {          i=k;        }
    else
    {          i=m;        }    } }
public class Main
{    public static void main (String args[])
  {       Test test1 = new Test(10);
    Test test2 = new Test(12, 15);
    System.out.println(test1.i);
    System.out.println(test2.i);
  } }
```

In the above program, The constructor Test is overloaded with another constructor. In the first call to the constructor, The constructor with one argument is called, and i will be initialized with the value 10. However, In the second call to the constructor, The constructor with the 2 arguments is called, and i will be initialized with the value 15.

---

34) What do you understand by copy constructor in Java?

There is no copy constructor in java. However, we can copy the values from one object to another like copy constructor in C++. There are many ways to copy the values of one object into another in java. They are:

By constructor
By assigning the values of one object into another
By clone() method of Object class
In this example, we are going to copy the values of one object into another using java constructor.

```
//Java program to initialize the values from one object to another
class Student6{
  int id;
  String name;
  //constructor to initialize integer and string
  Student6(int i,String n){
  id = i;
  name = n;
  }
  //constructor to initialize another object
  Student6(Student6 s){
  id = s.id;
  name =s.name;
  }
  void display(){System.out.println(id+" "+name);}

  public static void main(String args[]){
  Student6 s1 = new Student6(111,"Karan");
  Student6 s2 = new Student6(s1);
  s1.display();
  s2.display();
 }
}
```

Test it Now

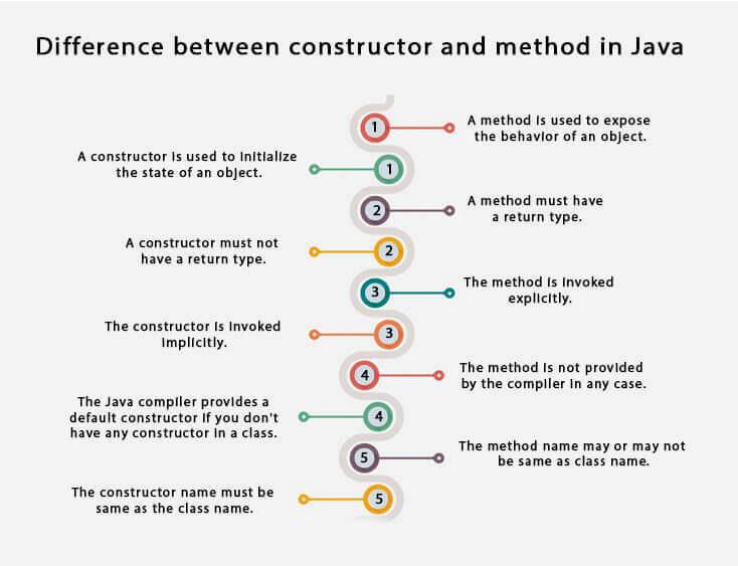
Output: 111 Karan 111 Karan

---

35) What are the differences between the constructors and methods?

There are many differences between constructors and methods. They are given below.

Java Constructor	Java Method
A constructor is used to initialize the state of an object.	A method is used to expose the behavior of an object.
A constructor must not have a return type.	A method must have a return type.
The constructor is invoked implicitly.	The method is invoked explicitly.
The Java compiler provides a default constructor if you don't have any constructor in a class.	The method is not provided by the compiler in any case.
The constructor name must be same as the class name.	The method name may or may not be same as class name.



36) What is the output of the following Java program?

```
public class Test
{  Test(int a, int b)  {
System.out.println("a = "+a+" b = "+b);
}
Test(int a, float b)  {  System.out.println("a = "+a+" b = "+b);  }
public static void main (String args[])  {
byte a = 10;   byte b = 15;
Test test = new Test(a,b);
}  }
```

The output of the following program is:
a = 10 b = 15
Here, the data type of the variables a and b, i.e., byte gets promoted to int, and the first parameterized constructor with the two integer parameters is called.

---

37) What is the output of the following Java program?

```
class Test
{     int i;   }
public class Main   {
   public static void main (String args[])      {
      Test test = new Test();          System.out.println(test.i);
   }  }
```

The output of the program is 0 because the variable i is initialized to 0 internally. As we know that a default constructor is invoked implicitly if there is no constructor in the class, the variable i is initialized to 0 since there is no constructor in the class.

---

38) What is the output of the following Java program?

```
class Test
{
   int test_a, test_b;
   Test(int a, int b)
   {
   test_a = a;      test_b = b;
   }
   public static void main (String args[])
   {
      Test test = new Test();
      System.out.println(test.test_a+" "+test.test_b);
   }
}
```

There is a **compiler error** in the program because there is a call to the default constructor in the main method which is not present in the class. However, there is only one parameterized constructor in the class Test. Therefore, no default constructor is invoked by the constructor implicitly.

Core Java - OOPs Concepts: static keyword Interview Questions

---

39) What is the static variable?

The static variable is used to refer to the common property of all objects (that is not unique for each object), e.g., The company name of employees, college name of students, etc. Static variable gets memory only once in the class area at the time of class loading. Using a static variable makes your program more memory efficient (it saves memory). Static variable belongs to the class rather than the object.

```
//Program of static variable

class Student8{
  int rollno;
  String name;
```

```
  static String college ="ITS";
     Student8(int r,String n){
  rollno = r;     name = n;
  }
 void display (){System.out.println(rollno+" "+name+" "+college);}

 public static void main(String args[]){
 Student8 s1 = new Student8(111,"Karan");
 Student8 s2 = new Student8(222,"Aryan");
  s1.display();   s2.display();
 }
}
Output:111 Karan ITS
     222  yan ITS
```

40. How many ways we can get the instance of Class class ?

There are 3 ways to get the instance of Class class. They are as follows:

- forName() method of Class class
- getClass() method of Object class
- the .class syntax

41. Java Concrete Classes?

A **concrete class** is a **class** that has an implementation for all of its methods. They cannot have any unimplemented methods. It can also extend an abstract**class** or implement an interface as long as it implements all their methods. It is a complete **class** and can be instantiated.

42. Volatile Keyword?

Volatile keyword is used to modify the value of a **variable** by different threads. It is also used to make classes thread safe. It means that multiple threads can use a method and instance of the classes at the same time without any problem.

Note: Volatile keyword ensure that the proper visibility of variable between threads.

Java Reflection

- How to create the javap tool ?
- How to create the appletviewer tool ?
- How to access the private method from outside the class ?
- How to create generic class and generic method in java ?
- What is annotation and how to create custom annotation ?
- What is the advantage of assertion and where we should not use it ?
- What is variable argument and what rules are defined for variable argument ?
- What is the difference between import and static import ?
- How autoboxing is applied in method overloading. Which concept beats autoboxing ?

- What is enum type and how to specify specific value to the enum constants ?

Exception Handling

- What is the difference between checked and unchecked exceptions?
- What happens behind the code int data=50/0;?
- Why use multiple catch block?
- Is there any possibility when finally block is not executed?
- What is exception propagation?
- What is the difference between throw and throws keyword?
- What are the 4 rules for using exception handling with method overriding?

New Features in Java

- How to create generic class and generic method in java ?
- What is annotation and how to create custom annotation ?
- What is the advantage of assertion and where we should not use it ?
- What is variable argument and what rules are defined for variable argument ?
- What is the difference between import and static import ?
- How autoboxing is applied in method overloading. Which concept beats autoboxing ?
- What is enum type and how to specify specific value to the enum constants ?

- What is Java?
- Various possible combination of try, Catch, Finally?
- Difference b/w Child p= new Parent(); or Parent p= new Child();
- Can we overload the main method?
- A Java Constructor returns a value but, what?
- Can we create a program without main method?
- What are the six ways to use this keyword?
- Why is multiple inheritance not supported in Java?
- Why use aggregation?
- Can we override the static method?
- What is the covariant return type?
- What are the three usages of Java super keyword?
- Why use instance initializer block?
- What is the usage of a blank final variable?
- What is a marker or tagged interface?
- What is runtime polymorphism or dynamic method dispatch?
- What is the difference between static and dynamic binding?
- How downcasting is possible in Java?
- What is the purpose of a private constructor?
- What is object cloning?

- **Topic Inner Classes Questions:**
  1. What is the internal code generated by the compiler for member inner class ?
  2. What are the two ways to create annonymous inner class ?
  3. Can we access the non-final local variable inside the local inner class ?
  4. How to access the static nested class ?
  5. Can we define an interface within the class ?
  6. Can we define a class within the interface ?
    1. How java manages Memory?
    2. Why is it advised to use hashcode and equals together?
    3. Comparable and Comparator in java
    4. How to create Singleton class in Java?
    5. Difference betwen equals and ==?
    6. When to use abstract class over interface?
    7. Difference between final, finally and finalize
    8. Why is it recommended to ues Immutable objects in Java

What is the difference between import and static import?

The import allows the java programmer to access classes of a package without package qualification whereas the static import feature allows to access the static members of a class without the class qualification. The import provides accessibility to classes and interface whereas static import provides accessibility to static members of the class.